

HERMES SMART CONTRACT SECURITY REPORT

Scan Date:	2026-07-06 14:57:45
Contract Address:	0x833589fcd6edb6e08f4c7c32d4f71b54bda02913
Analysis Mode:	Verified Original Source Code
Blockchain Network:	Base Blockchain (ChainID: 8453)

OVERALL SECURITY SCORE

78 / 100

1. Wallet & Token Distribution Ledger

- Top Holder 1 (Deployer Core): 42.15% of total supply (Centralization Risk Asset)
- Top Holder 2 (Liquidity Matrix Pool): 28.50% of total supply
- Top Holder 3 (External Locked Vault): 11.20% of total supply

2. AI Agent Autonomous Audit Ledger

- CENTRALIZATION WEIGHTS: The 'Deployer Core' holds 42.15% of the total supply, which exceeds the 25% threshold for significant centralization. This poses a risk where a single entity has substantial control over the token's ecosystem. (Deduction: 20 points)

- LOW/INFORMATIONAL: The contract `FiatTokenProxy` uses a floating compiler pragma `^0.6.2`, which can lead to unexpected behavior if compiled with a different Solidity version than intended, though less critical with minor versions. (Deduction: 2 points)

Security Audit Ledger:

1. Centralization Risk (SWC-104):

* Description: The "Deployer Core" wallet holds a significant portion (42.15%) of the total token supply. This level of concentration indicates a high degree of centralization, where a single entity has disproportionate control over the token's governance, supply, and potential for market manipulation. This could lead to concerns regarding censorship, rug pulls, or other malicious actions if the deployer's intentions change or its keys are compromised.

* Location: Token Distribution Summary

* Severity: Critical

* Deduction: 20 points

2. Floating Compiler Pragma (SWC-103):

* Description: The `Address.sol` file uses a floating compiler pragma `^0.6.2`. While `^` generally allows for compilation with newer patch versions, it can sometimes lead to unexpected compiler behavior or introduction of new bugs if the bytecode or ABI changes in a non-backward-compatible way in future minor releases. For production contracts, it is generally recommended to pin the pragma to a specific Solidity version (e.g., `pragma solidity 0.6.12;` for exactness). The `FiatTokenProxy.sol` uses a fixed pragma, which is good, but the imported library does not.

* Location: `@openzeppelin/contracts/utils/Address.sol`, line 3: `pragma solidity ^0.6.2;`

* Severity: Low/Informational

* Deduction: 2 points

The provided code for `FiatTokenProxy.sol` itself is a simple upgradeability proxy that inherits from `AdminUpgradeabilityProxy`, which is a well-established pattern. The `Address.sol` library, being an OpenZeppelin contract, is generally robust and widely audited. No critical, high, or medium vulnerabilities were found within the provided Solidity code snippets themselves. The primary deductions are due to the token distribution centralization and a minor informational finding related to the pragma in the imported library.